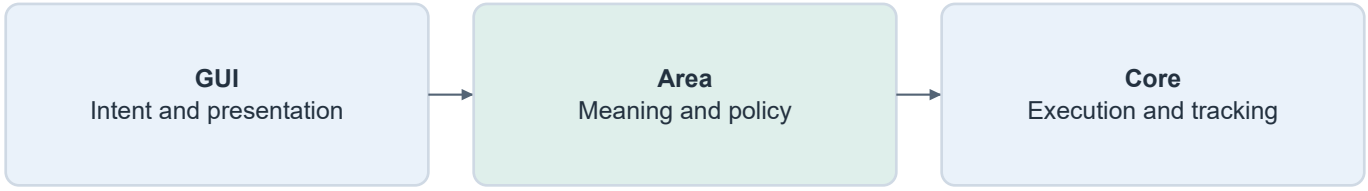
Core - Complete Map

LEONARDO V2

Scope, architecture, execution, ownership, lifecycle and integration. Targeted 28 October 2026 revision for accepted R01-R06 and Analysis composition. The 9 October edition evidence remains historical.

One runtime, clear ownership
Core determines how work runs, is tracked, observed and stopped. Areas determine what that work means and what may be saved.



One reference set

Written manual	Use
LIGHT_V2_RUNTIME.md	Scope and supported operational behaviour
CORE_ARCHITECTURE.md	Composition, dependencies and reconstruction
CORE_LIFECYCLES.md	Ordered workflows, failures and surviving state
CORE_DATA_CONTRACTS.md	Exact fields, APIs, validation and examples

Additive visual companion
This map supplements all four manuals. The July 2026 map remains unchanged as a historical reference; its growth recommendations are not current features.

Verified working candidate includes accepted uncommitted work. A commit identifier alone does not describe it. No architecture or persistence change is introduced by this edition.

Reading route and navigation

Read the entry manual first, then architecture, lifecycles and contracts. Use this map to follow the runtime from construction through shutdown.

Section	Page	Section	Page
Scope and authority	3	Production composition	4
Module dependency map	5	Threads and execution	6
Submission and delivery	7	Task lifecycle and history	8
Cancellation and publication	9	Ordered shutdown	10
Timeout, retry and restart	11	External processes	12
Connections and channels	13	Windows and guarded close	14
Actions and triggers	15	Audit and errors	16
Runtime Manager	17	Area integration	18
Models and persistence	19	Reconstruction and verification	20

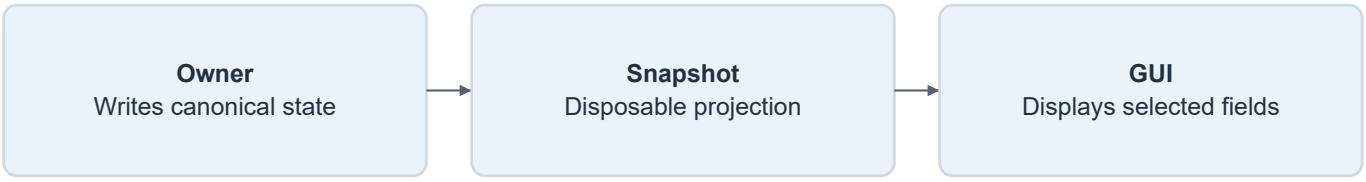
Current behaviour versus proposals
Solid flows describe implemented behaviour. Limitation callouts describe real boundaries. No new scheduler, global process supervisor, lifecycle framework or financial policy is proposed here.

Source authority: src/leonardo/core/, audit/event_v1.py and the GUI adapters identified in the manuals. Exact model and API inventories remain in CORE_DATA_CONTRACTS.md.

Scope and authority

State family	One authoritative owner
Application lifecycle	LeonardoApp
Loop, executor and admission	CoreRunner
Task identity, status, progress, history	TaskManager
Child identities, handles and exits	ProcessManager
Coarse connection/channel summaries	ConnectionRegistry
Logical window runtime records	WindowRegistry
Discoverable actions and triggers	ActionRegistry
Historical evidence / exception formatting	AuditLog / ErrorRouter

Outside Core
Market identity, OHLCV validity, financial formulas, saved Research objects, Database membership and publication policy stay with their canonical Area/store owners.



Production composition

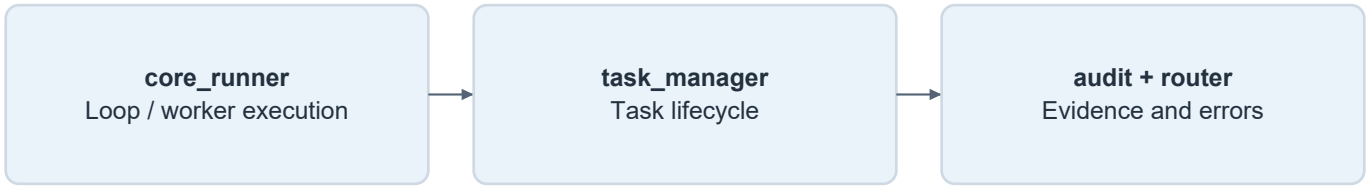


Construction group	Shared dependencies
Runtime foundation	Config, logger, audit/router, TaskManager, runner, processes, connections, windows
Connection and OHLCV	Provider registry, one store, market locks, download/maintenance, catalog and loader
Research and DMS inputs	Artifact, Environment, Recipe and creation stores/services reused across Areas
Analysis after DMS	AnalysisService(creation store), then AnalysisApplicationService(shared CoreRunner, Analysis domain)
Saved Research objects	Workspace/Notebook stores plus one shared link service
Inspection and UI context	Actions, RuntimeManagerBackend and frozen 21-field CoreContext; analysis_service is the final required reference

No hidden runtime
Background application work uses the shared CoreRunner. Windows and presenters receive existing services; they do not construct another executor or persistence authority.

The composition module imports Areas intentionally. Domain-neutral infrastructure managers do not take ownership of those Areas' rules.

Module dependency map



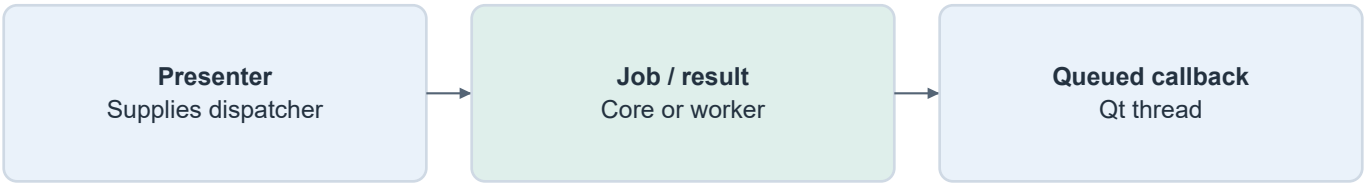
Module group	Role
config / async_runtime	Resolved paths, settings, name/coroutine helpers
process_manager	Launcher port and owned external handles
connection / window / action registries	Independent state families and projections
runtime_manager	Sequential readout from those same owners
app	Only application composition and lifecycle root
gui runner / tracker / presenters	Qt lifetime, accepted close and queued callback bridge

Dependency rule
CoreRunner and managers have no Qt or financial-service dependency. core/app.py assembles concrete Areas; its placement does not move their financial meaning into Core.

Reconstruction uses the 14-file Core/audit inventory. Runtime dataclasses and structural ports remain implementation models; only AuditEventV1 is the durable event schema.

Threads and execution placement

Thread / caller	Work performed
Qt GUI thread	Widgets, dialogs, selections, queued progress/result rendering
LeonardoCore daemon thread	One asyncio loop; coroutine jobs; task cancellation and terminal delivery
LeonardoWorker pool	Blocking/synchronous callables; four workers by default
External caller	Synchronous submit/cancel/shutdown waits; process handle APIs execute on their caller



Inline is not GUI-safe
Default progress callbacks execute on the job thread; default terminal callbacks execute on the Core loop. A GUI dispatcher must queue invocation to Qt.

The four-worker limit bounds concurrent worker execution, not queue length or task admission. Synchronous runner methods wait on the Core loop and must not block that loop by being called from its own callbacks.

Submission, progress and result

Method	Input / callback contract
submit_job	Async or synchronous callable receiving ProgressReporter; progress and result callbacks
submit_blocking_job	Synchronous callable in shared pool; rejects async functions; both callbacks
submit_coroutine	Actual coroutine object; result callback only, no progress_callback argument



1. Canonical task-name validation precedes private execution-wrapper allocation; public input/admission precedence is unchanged.

2. Submission returns after registration, not after the job finishes.

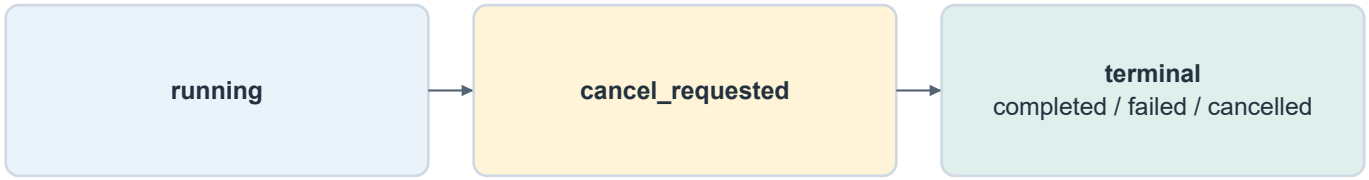
3. Reporter first updates TaskManager, then dispatches optional TaskProgress.

4. Task completion records status/time and emits terminal audit.

5. An established delivery waiter receives value, failure or cancellation and dispatches the result.

Meaning stays with the Area
Core accepts integer/None progress counters without inventing percentages or validating financial meaning. A timeout waiting for registration is not an automatic rollback guarantee.

Task lifecycle and retained history

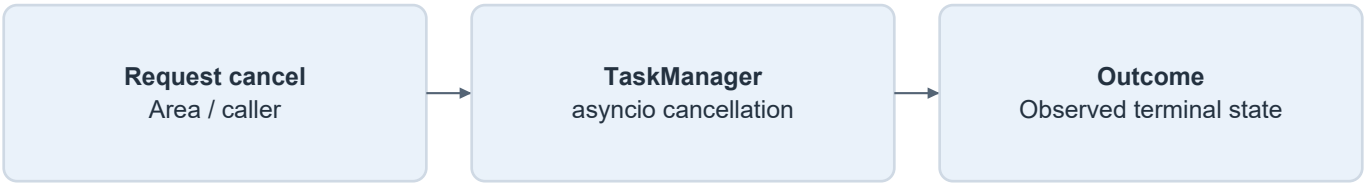


Normal success/failure can also go directly from running to terminal. A coroutine can catch cancellation and complete.

Invariant	Current rule
Identity	UUID hex; normalized names reject active duplicates unless permitted
History cap	500 by default; retention, not task-admission limit
Eligible eviction	Creation and completion scan past active records to remove oldest terminal history
Active overflow	Preserve every active record even above the cap
Waiters and callbacks	Established task reference survives history eviction
Late lookup	New wait after eviction may raise KeyError; snapshot lookup returns None

No data deletion
Runtime history eviction does not delete Notebooks, Workspaces, Artifacts, Databases or audit records.

Cancellation and publication boundaries



Worker survival is real
Cancelling an asyncio awaiter cannot forcibly terminate the Python callable already running in a worker. Core executor shutdown waits for that callable to settle.

Responsibility	Owner
Runtime cancel request and status	TaskManager, scheduled through CoreRunner
Cooperative worker checks	Area job/event/checkpoints
Publication boundary and late-cancel refusal	Area/store workflow, including DMS gates
Disposed view or changed selection	Presenter/service lifetime and stale-result fences
Saved object survival	Canonical persistence and relationship owners

Closing one Suite is not closing the shared runtime. Other windows can still submit work. Both app shutdown routes request DMS, then Analysis cancellation before Core shutdown. Analysis Suite retirement is fenced by window/presenter identity and ends its session only.

Ordered application shutdown

1. LeonardoApp preserves whether status was already failed; otherwise enters stopping.

2. Request pending DMS cancellation, then Analysis cancellation, through their existing services.

3. Core closes admission and creates or reuses its cancellation future.

4. Request loop stop once; cancel/gather remaining async work, close async generators, drain workers, close loop and join thread.

5. Terminate/poll managed child processes; escalate if necessary.

6. Mark connection summaries disconnected and remaining window records closed.

7. Emit application.stopped; set stopped only if not preserving an earlier failed status.

8. Close audit sinks in finally, including error paths.

Failure interrupts the sequence
An exception sets app status failed, routes evidence and re-raises. Later ordered steps may still be pending. Successful cleanup retry does not erase the earlier failed status.

Core-only stop uses DMS -> Analysis -> Core without the later app cleanup. Timeout remains per-wait, not a total deadline; tracking does not delete data or close provider sockets.

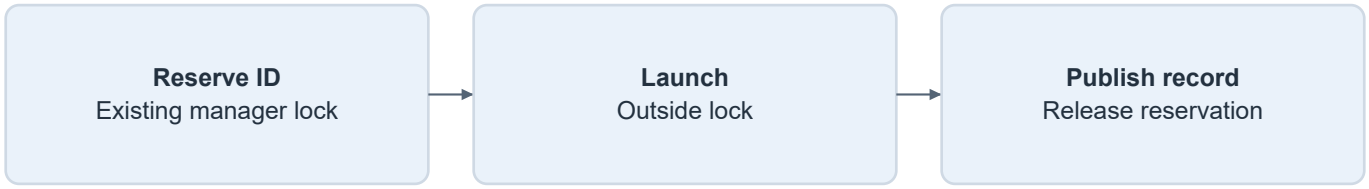
Timeout, retry and explicit restart

Observation	Required behaviour
Cancellation still pending	Reuse one future; retry may time out again
Executor still draining	Do not inject a second stop; reject start/submission
Thread alive at wait deadline	Raise honest TimeoutError
Thread exited, loop remains unclosed	Report incomplete cleanup; refuse start
Thread exited and loop closed	Retire only matching lifecycle; repeated shutdown succeeds
Later explicit Core restart	Create fresh loop/pool after completion; earlier caller cannot clear it
Earlier application failure	Cleanup may finish while app status stays failed

Two different restart boundaries
A fully stopped CoreRunner supports explicit restart. A normally stopped LeonardoApp requires fresh application composition. Neither rule creates automatic recovery.

Audit controls held workers with events, exercised retry both during and after drain, checked overlapping callers, and injected a finalization failure. No forced worker kill or replacement lifecycle owner was introduced.

External-process ownership

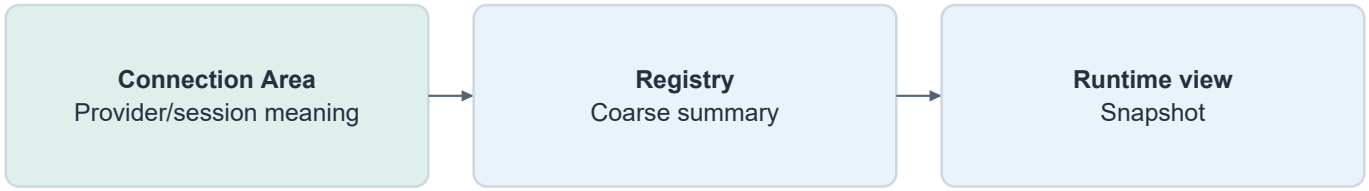


Event	State and evidence
Concurrent duplicate	Rejected before its launcher executes
Launch failure	Route exception; reservation released for retry
Exit detected by poll	Zero -> completed; nonzero -> failed; requested termination -> terminated
Repeated terminal poll	One terminal audit event per record
Shutdown	Terminate, poll until timeout, kill remaining, wait up to min(timeout, 1 second)
After exit	Keep terminal record for manager lifetime; no generic unregister/reload

Bounded concurrency correction
Different IDs may launch concurrently. In-flight ID reservation is not a new global admission policy for launches racing shutdown.

Historical downloads and Research calculations are tasks in the current process, not ProcessManager children. An empty Processes tab can be correct.

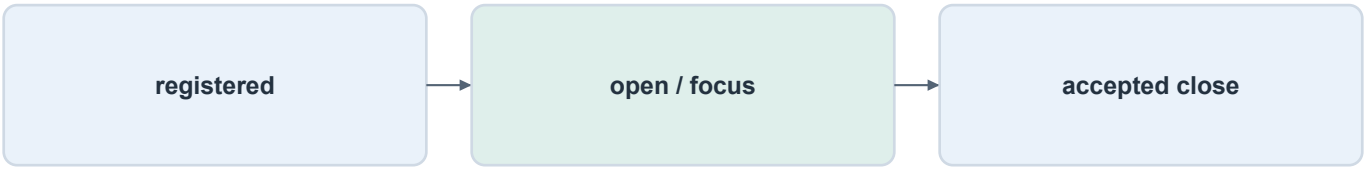
Connections and channel summaries



Surface	Current semantics
Statuses	registered, connecting, connected, degraded, disconnect_requested, disconnected, failed
Transitions	Mark methods assign states; no strict predecessor state machine
Heartbeat / errors	Aware UTC timestamp; connected clears prior error
Channels	Existing parent, unique channel ID; received/sent/error counters with positive integer increments
Shutdown tracking	Mark non-failed/non-disconnected summaries disconnected; retain records/counters
Current production	Historical Bybit REST summaries; no production websocket workflow

No socket ownership here
Core tracking does not authenticate, reconnect, subscribe or close an actual network session. Detailed provider lifecycle remains in the Connection Area.

Windows and guarded closure

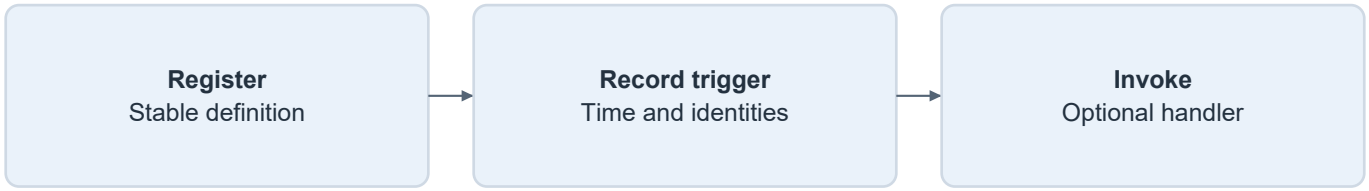


Operation	Meaning
Register	Same ID/title/type is idempotent; conflicting identity registration rejects
Show / focus	Open record; focus requires open
Guarded main / Research close	close_on_event=False; accepted closed signal records closure
Dirty-change Cancel	Preserve actual window and open registry state
Reopen	Same logical ID; preserve first opened_at, clear closed_at
Registry close	Does not itself close/delete QWidget or saved Area objects

Identity is not the title
Distinct editor modes need distinct logical IDs. Closing a view, removing an assignment and deleting a saved object are different operations.

Ordinary tracked windows can use close-event recording. Guarded windows require their accepted-close path; hiding a widget is not a close event in this adapter.

Discoverable actions and triggers

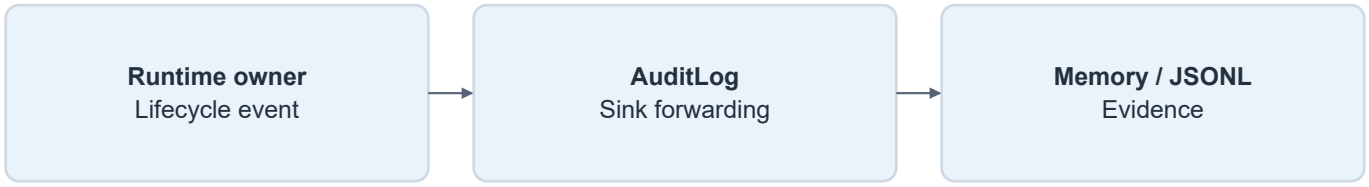


Contract	Current behaviour
Definition	ID, label, optional handler/window, risk, confirmation, audit policy, metadata
Duplicates	Exact definition is idempotent; conflict rejects
Retention	Definitions remain; latest 500 trigger records
Invocation	Trigger is recorded before synchronous handler execution
Failure	No automatic retry/exception wrapper around handler
Risk and confirmation	Descriptive metadata; Area/UI must enforce actual policy

A trigger is not a success receipt
action.triggered proves invocation intent was recorded, not that a handler completed or a persistent mutation was authorized.

Local display controls need not become global actions. A registered action is not automatically a task; long-running work still enters through its Area service and CoreRunner.

Audit storage and error routing



Component	Semantics
Memory sink	Bounded deque, default 1000; close/flush no-op
JSONL sink	Opt-in, lazy append, flush each event; close can later reopen
Composite emit	Try all; record failures; require at least one delivery
Composite snapshot	First non-empty successful sink, not a merged history
ErrorRouter	error.reported with error type/message and identity/context; no retry or dialog
Durable schema	AuditEventV1; runtime records are not loaded from the audit log

Failure handling is explicit
No blanket audit-is-harmless promise: ordinary task/process emission boundaries log sink errors, while direct app/action/router calls retain their actual propagation semantics.

In JSONL mode, application composition orders JSONL before memory. Backend sink failures are inspectable but have no separate Runtime Manager tab.

Runtime Manager: backend and GUI

Visible tab	Columns
Tasks	ID, task name, status, progress message
Processes	ID, label, status, PID
Connections	ID, label, status, protocol
Windows	ID, title, status, type
Actions	ID, label, window, risk
Audit	Timestamp, severity, event, message

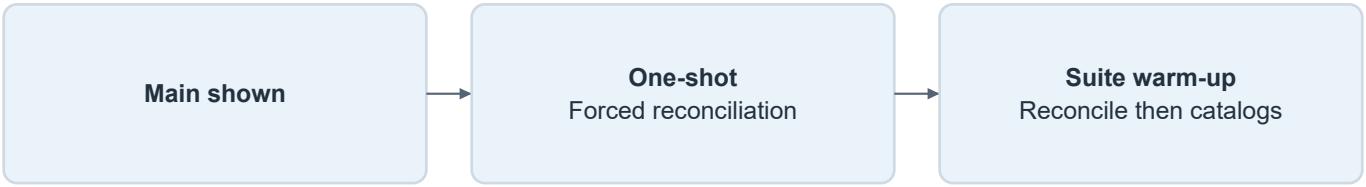
Backend contains more
RuntimeSnapshot also carries generated time, app/actor, websocket channels and audit-sink failures. There are six visible tabs, not separate channel/failure views.

Polling remains authoritative
Refresh calls ProcessManager.snapshots, which polls and may record a terminal exit and audit event. The GUI has no mutation controls; snapshot creation is not globally atomic or side-effect-free.

Rows are disposable projections, not parallel registries. Action projection omits handlers and arbitrary metadata. RuntimeSnapshot includes Python datetime values and is not a durable JSON schema.

Integration with current Areas

Area	Core work / retained authority
Connection + OHLCV	Preflight, download, validation and maintenance tasks; provider semantics, canonical store, validator and locks remain in their owners
Research	Dataset/study/Environment/Workspace/Notebook tasks; chart state, compatibility, assignments and persistence stay in Research
Data Manager	Catalog, reconciliation, creation and explicit update/publication tasks; source-currentness and publication policy stay in DMS
Analysis	Saved-database binning, patterns, pinned refinement, inspection and sensitivity; shared Core and creation store
GUI	Queued callbacks, current selection, dirty decisions and disposal fences; no financial calculations in widgets



Suite opening can occur later. Research dataset terminal bookkeeping runs before outward delivery; callbacks dispatch once outside its lock, even if a retired GUI discards them.

Correct current DMS description
The coordinator is one-shot, not a recurring 60-second timer. Later refresh is explicit/event-driven. The retained database-rebuild endpoint refuses under current policy; it is not a supported workflow.

Models, paths and persistence

Category	Meaning
Configuration	RuntimePaths, AuditConfig, AppConfig; default config does not create directories
Context	21 required references; final analysis_service: AnalysisApplicationService; no independent mutable authority
Execution models	TaskSubmission, TaskProgress, TaskResult and owner snapshots
Internal records	Five private dataclasses; not a durable/public compatibility promise
Durable event	AuditEventV1: 17 fields, aware UTC time, normalized JSON-compatible details
Inventory	22 dataclasses / 111 source-declared callable-property-constructor signatures from 14 source files

Default persistence boundary
Core runtime histories are memory-only. Optional audit goes to runs/audit.jsonl. OHLCV, Environments, Workspaces, Notebooks and DMS files are written by their own stores.

Frozen dataclasses are not deeply immutable. AuditEventV1 validates text/time/details, but does not enforce a severity enum or reject every non-1.0 nonempty version. Contracts document exact enforcement rather than stronger inferred guarantees.

Reconstruction and verification

1. Build event normalization, config, sinks and ErrorRouter.

2. Implement TaskManager, then one-loop CoreRunner and shared workers.

3. Implement process ownership and independent registries.

4. Aggregate snapshots directly from owners; preserve polling semantics.

5. Compose the existing Areas/stores in LeonardoApp.

6. Attach GUI runner, queued delivery and accepted-close tracking.

7. Verify contracts, both shutdown retry timings, history pressure, identity admission and native generated-data close/reopen.

Historical 9 October code evidence	Result
Core V2 audit checks	42 passed, including six additional audit probes
Full repository evidence	2,845 passed; seven Windows symbolic-link privilege skips
Core/schema/selected GUI gate	161 passed
Native Windows smoke	10 checks passed; generated data only

These are original 9 October results, not R07 executions. Accepted R01-R06 receipts remain separate. R07 checks documents only; R08 integrated and R09 native acceptance are pending.

Stop at the verified boundary
No scheduler, storage change, new lifecycle framework or optional improvement is introduced. The map and manuals describe the accepted current program. Human visual/product acceptance remains separate.